
MasonBook

EIYM / Mason Protector - Operator Manual

Build Edition 2026.04

.NET Framework 4.7.2 - WinExe - AnyCPU

Author: Mason

*This edition supersedes all prior MasonBook revisions.
VM unified - Debug surface removed - Codebase comment-stripped*

Table of Contents

1. Foreword	About this build, what changed, who it is for.
2. Architecture overview	Pipeline, settings model, runtime layout.
3. Build and run	MSBuild target, output binary, first launch.
4. The Builder UI	How to load a target, choose protections, build.
5. Protection catalogue	All protections grouped by category, with flags.
6. VM obfuscation (V1 + V2 unified)	Unified file, two engines, eligibility rules.
7. Encryption layer	Strings, ints, fields, methods, arrays, runtime.
8. Obfuscation layer	Control flow, calli, proxies, virtualization.
9. Anti-* layer	Debug, tamper, dump, hook, http, vm, ildasm, de4dot.
10. Stealth layer	Renamer, junk, watermark, metadata, dnSpy crasher.
11. Operational notes	Anti* + redirected I/O, AnyCPU, pdb handling.
12. Test harness	TestApp probes that ship with the project.
13. Changelog for this edition	VM merge, debug surface removal, comment strip.
14. Appendix A - Settings reference	Every flag in ProtectionSettings.
15. Appendix B - File map	Where each protection lives in the source tree.

1. Foreword

MasonBook is the operator manual for the EIYM / Mason Protector. It documents the tool as it ships in this build edition, not an aspirational design. Every protection listed here corresponds to a class compiled into **EIYM.exe**, every flag corresponds to a property of **MasonProtector.Core.ProtectionSettings**, and every behaviour described in the operational notes has been observed in the test harness that ships alongside the source tree.

This edition introduces three structural changes from earlier MasonBook drafts. First, the two virtual-machine engines (the integer-stack VM and the object-stack VM) now live in a single source file, **VMObfuscation.cs**, which exposes both **VMObfuscationProtection** and **VMObfuscationV2Protection**. Second, the diagnostic / debug surface (the Builder Debug button, the DebugProtect wrapper, the leak-diagnostic writer and the .eiy-diag.txt artefact) has been fully removed - the tool now ships as a single-purpose protector. Third, the entire compiled C# source has been stripped of comments to keep the codebase intentionally opaque to drive-by reading; functional headers and brand markers are retained where they are part of the artefact (for example the unified VM file header).

Who this manual is for

Operators who build protected binaries for their own .NET assemblies; developers extending the protection pipeline; reviewers who need to map a runtime artefact back to the source that produced it.

2. Architecture overview

The protector loads a target assembly with **dnlib**, walks every module / type / method, and applies a sequence of transformations driven by a single **ProtectionSettings** object. Each transformation is a self-contained class under **Core/Protections/** that takes the engine as a context, mutates the IL or metadata in place, and returns. The order in which transformations run is fixed by **Obfuscation.cs**; flags do not reorder the pipeline, they only enable or disable individual stages.

Pipeline phases

- **Pre-analysis** -- load module, build symbol map, capture pdb references.
- **Encryption** -- strings, ints, constants, fields, arrays, method bodies, delegate dispatch, cross-references, runtime-deferred decoding, polymorphic.
- **Obfuscation** -- control flow, opaque predicates, calli conversion, local-to-field, reference and call hiding proxies, branch confusion, stack underflow, method scattering, method inliner, numeric obfuscation, control-flow flattening (variant 2), code virtualization, and the two VM engines (V1 int-stack and V2 object-stack).
- **Anti*** -- debug, tamper, dump, hook, http, vm, ildasm, de4dot, memory dump.
- **Stealth** -- renamer, junk code, watermark, type scrambler, hide methods, fake attributes, token confusion, metadata confusion, invalid metadata, dnSpy crasher, entry-point mover.
- **Emit** -- write the protected module to disk; pdb is dropped.

Settings model

The Builder UI binds every checkbox to one bool property on **ProtectionSettings**; the same object is what tests construct programmatically. There is no ambient configuration -- if a flag is not on the settings object, the protection cannot be enabled.

3. Build and run

Toolchain

- Target: **.NET Framework 4.7.2**, OutputType **WinExe**, AnyCPU.
- Project file: **MasonProtector\MasonProtector.csproj**.
- External reference: **dnlib 4.5.0** (NuGet, restored under **packages/**).
- Built artefact: **MasonProtector\bin\Debug\EIYM.exe**.

Building from the command line

MSBuild from a .NET Framework toolset (the v4.0.30319 toolset is enough; the project carries ToolsVersion 15.0 but degrades cleanly):

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe  
MasonProtector\MasonProtector.csproj -p:Configuration=Debug
```

First launch

Run **EIYM.exe**. The Builder window opens. Drop the target assembly onto the form (or use the file picker), tick the protections you want, click **Build**. The protected file is written next to the input with a **_protected** suffix.

4. The Builder UI

The Builder is a single Windows Forms surface defined in **Builder.cs** / **Builder.Designer.cs**. It exposes a checkbox for every protection plus a small number of integer / string knobs (rename length, prefix, alphabet, junk count). The form has exactly one action button -- **Build** -- which spawns the protection pipeline on a worker thread and writes log lines back to the status pane.

What is no longer in the UI

Earlier builds shipped a second action button labelled **Debug** that ran a side pipeline writing **.eiym-diag.txt** next to the output. That button, its click handler, the DebugProtect wrapper inside Obfuscation.cs, the RunLeakDiagnostic method and the diagnostic file writer have all been removed. **EIYM.exe** now has a single action surface: build a protected binary, write the binary, exit. Operators who need diagnostic output should run the protector under their own harness; the test scaffolding under **TestApp/** is the reference.

5. Protection catalogue

The full set of protections compiled into this build, grouped by the subfolder under **Core/Protections/** in which they live. The third column is the exact **ProtectionSettings** property name.

Category	Protection	Setting flag
Encryption	String encryption	StringEncryption
Encryption	Integer encoding	IntEncoding
Encryption	Constants encoding	ConstantsEncoding
Encryption	Field encryption	FieldEncryption
Encryption	Array encryption	ArrayEncryption
Encryption	Method body encryption	MethodBodyEncryption
Encryption	Cross-reference encryption	CrossReferenceEncryption
Encryption	Polymorphic encryption	PolymorphicEncryption
Encryption	Runtime encryption	RuntimeEncryption
Encryption	Mutation encoding	MutationEncoding
Encryption	String composition	StringComposition
Encryption	Delegate encryption	DelegateEncryption
Encryption	Body vault (runtime)	(internal, used by other passes)
Encryption	Type cloner (helper)	(internal)
Obfuscation	Control flow	ControlFlow
Obfuscation	Control-flow flattening v2	ControlFlowFlattening2
Obfuscation	Opaque predicates	OpaquePredicates
Obfuscation	Proxy calls	ProxyCalls
Obfuscation	Reference proxy	ReferenceProxy
Obfuscation	Call hiding	CallHiding
Obfuscation	Calli conversion	CalliConversion
Obfuscation	Local-to-field	Local2Field
Obfuscation	Stack underflow	StackUnderflow
Obfuscation	Method scattering	MethodScattering
Obfuscation	Method inliner	MethodInliner
Obfuscation	Numeric obfuscation	NumericObfuscation
Obfuscation	Branch confusion	BranchConfusion
Obfuscation	Code virtualization	CodeVirtualization
Obfuscation	VM obfuscation (V1 int-stack)	VMObfuscation
Obfuscation	VM obfuscation (V2 object-stack)	VMObfuscationV2
Anti	Anti-debug	AntiDebug
Anti	Scatter anti-debug	(linked into AntiDebug)
Anti	Anti-tamper	AntiTamper
Anti	Anti-dump	AntiDump

Category	Protection	Setting flag
Anti	Anti-memory-dump	AntiMemoryDump
Anti	Anti-hook	AntiHook
Anti	Anti-http	AntiHttp
Anti	Anti-VM	AntiVM
Anti	Anti-de4dot	AntiDe4dot
Anti	Anti-ILDasm	AntiILDasm
Stealth	Renamer	EnableRenamer (+ RenameTypes / RenameMethods / RenameFields / RenameProperties / RenameEvents / RenameNamespaces)
Stealth	Hide methods	HideMethods
Stealth	Junk code	JunkCode (+ JunkCount)
Stealth	Fake attributes	FakeAttributes
Stealth	Watermark	Watermark
Stealth	Type scrambler	TypeScrambler
Stealth	Token confusion	TokenConfusion
Stealth	Metadata confusion	(internal, paired with TokenConfusion)
Stealth	Invalid metadata	InvalidMetadata
Stealth	DnSpy crasher	DnSpyCrasher
Stealth	Entry point mover	EntryPointMover

6. VM obfuscation (V1 + V2 unified)

The two virtual-machine engines now ship in a single source file, **Core/Protections/Obfuscation/VMObfuscation.cs**. The file opens with the marker:

```
// EIYM / Mason Protector
// VMObfuscation (V1 int-stack + V2 object-stack)
// (c) Mason
```

Inside the same namespace block (**MasonProtector.Core**) the file declares two classes, side-by-side:

- **VMObfuscationProtection** -- the original int-stack VM. Translates eligible IL into a custom byte-coded program that runs on a stack of 32-bit integers. Used for arithmetic, bitwise, comparison, branch and locals/args traffic.
- **VMObfuscationV2Protection** -- the object-stack VM. Generalises V1 to handle reference types, string operations, newobj, virtual / interface dispatch and ref-shape signatures. Methods that V1 would refuse on signature grounds typically become V2-eligible.

Eligibility

Both engines walk the method list, score each method against their eligibility rules, and skip anything they cannot prove safe. V1 prefers narrow integer methods; V2 prefers methods whose surface is wider but where every operand can be boxed onto the object stack. The two engines do not double-protect the same method -- once a method is virtualized by one engine, it is excluded from the candidate set of the other.

Verification

Three probes that ship under **TestApp/** exercise the merged file:

- **V2NewobjProbe.exe** -- compiles a target whose methods do **newobj** with ref-shape signatures, applies V2-only protection, runs the result.
- **V1V2Probe.exe** -- compiles a target with both integer-only methods and string-returning methods, applies V1+V2 together, asserts both shapes still produce the expected output.
- **AllOnExceptAntiTest.exe** -- 30 randomised target assemblies, every non-Anti protection enabled including both VM engines, every binary must exit zero with the expected stdout.

Latest run of all three: **PASS, PASS, 30 / 30**.

7. Encryption layer

Encryption transformations replace static data and method bodies with deferred-decoding equivalents. Each pass is independent; enabling all of them stacks the layers.

- **StringEncryption** -- every literal string is replaced by a call to a generated decoder; the decoder reads from an embedded blob keyed per-method.
- **IntEncoding** -- numeric constants are encoded with a per-call transform whose inverse is generated inline.
- **ConstantsEncoding** -- generalises IntEncoding to other constant loads (long, float, double, bool).
- **FieldEncryption** -- static and instance field traffic is rerouted through accessors that decode on read and encode on write.

- **ArrayEncryption** -- byte/char arrays embedded in IL (the canonical **RuntimeHelpers.InitializeArray** shape) are XOR-masked at write and unmasked on first read through a generated initialiser.
- **MethodBodyEncryption** -- selected method bodies are stored encrypted and reconstructed at runtime through the body vault.
- **CrossReferenceEncryption** -- token references between the protected module's own types are rerouted through a runtime indirection table.
- **PolymorphicEncryption** -- protected payloads are wrapped in a tiny polymorphic decoder that mutates between builds.
- **RuntimeEncryption** -- defers decoding of hot constants to first use, rather than module-load time.
- **MutationEncoding** -- weaves additional arithmetic mutations into the decoder bodies so two identical-looking encoders are not actually identical IL.
- **StringComposition** -- breaks selected strings into pieces that are re-assembled by the decoder; helps against straight-line decoding scrapers.
- **DelegateEncryption** -- routes selected calls through a delegate vended by a runtime helper so the call target is not visible at the call site.

8. Obfuscation layer

Obfuscation transformations rewrite IL shape without changing observable behaviour. Several stages depend on each other: opaque predicates feed branch confusion, control flow feeds the VMS, calli conversion lowers method tokens to indirect calls.

- **ControlFlow** -- classical block-shuffle with linker stubs.
- **ControlFlowFlattening2** -- second-generation flattener; turns the method body into a state-machine dispatch loop.
- **OpaquePredicates** -- inserts always-true / always-false checks whose values are computed from generated constants, defeating naive dead-code elimination.
- **ProxyCalls** -- internal direct calls become indirections through generated stubs.
- **ReferenceProxy** -- like ProxyCalls but for token references and field accesses.
- **CallHiding** -- dispatch is renamed and re-grouped so reflection-style scanners cannot pattern-match on the obvious call shapes.
- **CalliConversion** -- direct call sites become **calli** with a function pointer loaded from a generated table.
- **Local2Field** -- promotes selected locals to module-level fields; perturbs the disassembler's local-frame inference.
- **StackUnderflow** -- introduces patterns that look like underflow to naive verifiers but balance correctly under a real stack walker.
- **MethodScattering** -- breaks long bodies into trampolined fragments across multiple synthetic methods.
- **MethodInliner** -- selectively inlines tiny callees to disrupt name-based call-graph analysis.
- **NumericObfuscation** -- algebraic rewrites of arithmetic chains.
- **BranchConfusion** -- replaces br / brtrue / brfalse with patterns that compute the branch outcome from earlier-staged predicates.

- **CodeVirtualization** -- a coarse virtualizer applied before V1/V2; captures whole-method shapes that the VMs do not target.
- **VMObfuscation** -- V1 int-stack VM (see section 6).
- **VMObfuscationV2** -- V2 object-stack VM (see section 6).

9. Anti-* layer

Anti-* protections are runtime checks injected into the protected assembly's module initializer (and a small number of hot paths). They terminate the process when a hostile environment is detected. They are intentionally aggressive.

- **AntiDebug** -- managed and native debugger probes; backed by **ScatterAntiDebug** which spreads checks across multiple call sites.
- **AntiTamper** -- module-image hash check at startup; mismatch terminates.
- **AntiDump** -- header sanity, scrambled section pointers; defeats simple full-process dumpers.
- **AntiMemoryDump** -- companion to AntiDump targeting in-memory snapshot tools.
- **AntiHook** -- prologue-byte verification on critical .NET runtime methods.
- **AntiHttp** -- detects local HTTP MITM (Fiddler / mitmproxy) on standard ports / proxy env.
- **AntiVM** -- well-known VM artefact lookup (vendor strings, devices, MAC OUI).
- **AntiDe4dot** -- token / metadata patterns that crash de4dot.
- **AntillDasm** -- emits the SuppressIldasmAttribute and shapes metadata so ildasm refuses to walk it.

10. Stealth layer

- **Renamer** -- gated on **EnableRenamer**; per-kind toggles (**RenameNamespaces**, **RenameTypes**, **RenameMethods**, **RenameFields**, **RenameProperties**, **RenameEvents**); alphabet and length come from **RandomChars** / **RenameLength**; **RenamePrefix** can pin a stable prefix.
- **HideMethods** -- adds the **CompilerGenerated** / **DebuggerHidden** / **DebuggerStepThrough** triple to selected methods.
- **JunkCode** -- inserts **JunkCount** additional dead methods / branches that look plausible.
- **FakeAttributes** -- decorates types and methods with attributes that lead automated triage tools toward false conclusions.
- **Watermark** -- writes a deterministic, brand-tied marker into metadata; the operator cannot disable the brand identity.
- **TypeScrambler** -- reorders type / member layout to break source-order assumptions.
- **TokenConfusion** -- reshapes the token table so token-based heuristics misfire.
- **InvalidMetadata** -- emits malformed-but-loadable metadata (oversized table padding, unused-but-claimed streams).
- **DnSpyCrasher** -- patterns specifically tuned to crash dnSpy parsers.
- **EntryPointMover** -- the visible **Main** is replaced by a trampoline; the real entry is reached through the runtime indirection.

11. Operational notes

Anti* and redirected I/O

When every Anti* flag is on, the protected exe will terminate if its parent process is redirecting stdin / stdout / stderr -- that is one of the patterns AntiDebug treats as hostile. This is intentional. If a host harness needs to capture stdout, it must launch the protected exe in a configuration the Anti checks accept (no redirection, no debugger, no hook), or it must protect the target with Anti* turned off. The reference test for the full non-Anti pipeline is **AllOnExceptAntiTest.exe**, which runs 30 randomised binaries with every non-Anti flag on; the baseline expectation is **30 / 30**.

AnyCPU and bitness

EIYM.exe is AnyCPU. Targets of either bitness can be protected. The protector itself is single-threaded per build and holds the input module open through the dnlib writer; do not race two builds against the same input.

PDB handling

PDB references are dropped during emit. Protected output never carries a matching PDB even if the input had one.

No diagnostic side-channel

Earlier builds wrote a **.eiym-diag.txt** companion file next to the output when the now-removed Debug action was triggered. The current build writes no such file under any code path; if you find a stale one, it is from a prior run.

12. Test harness

The **TestApp/** tree contains a small set of probes -- single-purpose C# console apps that load EIYM.exe by reflection, build a tiny target, apply a known set of flags, run the protected output and assert. The three probes that gate every release are:

- **V2NewobjProbe** -- VM V2 must accept **newobj** ref-shape methods.
- **V1V2Probe** -- both VMs cooperate on the same target.
- **AllOnExceptAntiTest** -- 30 random targets, full non-Anti pipeline, expect 30/30.

The harness is intentionally minimal -- only the probes needed to gate a release plus the **TestApp.cs** input target they consume. Stale diagnostic exes and dump folders from older debug-feature runs have been removed in this edition.

13. Changelog for this edition

VM unification

VMObfuscation.cs and **VMObfuscationV2.cs** have been merged into a single file, **VMObfuscation.cs**, holding both engines under the same namespace. Duplicate **using** declarations were deduplicated, the dnlib aliases (**DnFieldAttributes**, **DnMethodAttributes**, **DnMethodImplAttributes**, **DnTypeAttributes**, **DnOpCodes**) are declared once at the top, and the **.csproj** entry for **VMObfuscationV2.cs** was removed. Probe results post-merge: V2NewobjProbe PASS, V1V2Probe PASS, AllOnExceptAnti 30/30.

Debug surface removed

The Builder Debug button, the **MASON_btnDebugStart_Click** handler and its enable / disable plumbing in the build finally-block, the **DebugProtect** wrapper inside **Obfuscation.cs**, the **RunLeakDiagnostic** method, the side-pipeline's **.eiym-diag.txt** writer, the **Core/DebugEngine.cs** source file and the **Core/Engine/ProtectionResult.cs** source file have all been removed. The **.csproj** no longer references either deleted source. The Builder is now a single-action UI.

Codebase comment-strip

Every **.cs** source compiled into **EIYM.exe** has been passed through a tokenizer-aware stripper that removes both line (**//**) and block (**/* ... */**) comments while preserving regular, verbatim (**@"..."**) and interpolated (**\$"..."**) string literals, plus char literals. The stripper is **_tools/strip_comments.py**; it is driven by the **<Compile Include=>** list in the project file so it touches exactly the 62 files MSBuild compiles and nothing under **obj/**, **bin/**, or sibling projects. The unified VM file's three-line header marker is preserved (it is part of the artefact, not a developer comment). Total comment volume removed this pass: ~42 KB.

Project tree cleanup

Stale and unreferenced files removed from the repo: a 165 MB nested copy of an old source tree (**MasonProtector/MasonProtector/**), four **.rar** archives, seven prebuilt DLLs at the project root that the **.csproj** never referenced, a duplicate **.sln**, two stale builder config text files, the duplicate **packages/** folder, the empty **Core/Analysis/** and **Core/Crypto/** folders, and 664 stale TestApp diagnostic artefacts (**.eiym-diag.txt** + dump folders + one-shot probe exes). Repo size dropped from ~225 MB to ~27 MB. Dead code in **PreAnalysis.cs** (unread fields **HasPInvoke**, **HasCOMInterop**, **HasWPF**, **HasWinForms**, **HasMEF**, **HasASPNet**; unused method pairs **ShouldExcludeFromRename**, **ShouldExcludeMethodFromRename**; the **ConflictMap / Warnings** machinery) was deleted, and an empty branch in **Obfuscation.ResolveConflicts** was collapsed.

Build verification

Post-cleanup rebuild via **MSBuild.exe -p:Configuration=Debug** succeeds with no warnings beyond the standard ToolsVersion-15 fallback notice. All three gating probes still pass.

14. Appendix A - Settings reference

Every property exposed by **MasonProtector.Core.ProtectionSettings**, in declaration order, with type and default.

Property	Type	Default
EnableRenamer	bool	false
RenameNamespaces	bool	false
RenameTypes	bool	false
RenameMethods	bool	false
RenameFields	bool	false
RenameProperties	bool	false
RenameEvents	bool	false
RenameLength	int	12
RenamePrefix	string	""
RandomChars	string	A-Z a-z (alphabet)
StringEncryption	bool	false
IntEncoding	bool	false
ControlFlow	bool	false
AntiDebug	bool	false
AntiVM	bool	false
JunkCode	bool	false
JunkCount	int	50
ResourceProtection	bool	false
AntiDump	bool	false
AntiTamper	bool	false
AntiDe4dot	bool	false
ProxyCalls	bool	false
HideMethods	bool	false
FakeAttributes	bool	false
Watermark	bool	false
VMObfuscation	bool	false
VMObfuscationV2	bool	false
CalliConversion	bool	false
Local2Field	bool	false
MutationEncoding	bool	false
RuntimeEncryption	bool	false
ConstantsEncoding	bool	false
OpaquePredicates	bool	false
ReferenceProxy	bool	false
CallHiding	bool	false
MethodBodyEncryption	bool	false
AntiILDasm	bool	false
TokenConfusion	bool	false

Property	Type	Default
FieldEncryption	bool	false
StackUnderflow	bool	false
CrossReferenceEncryption	bool	false
PolymorphicEncryption	bool	false
StringComposition	bool	false
MethodScattering	bool	false
DelegateEncryption	bool	false
ControlFlowFlattening2	bool	false
InvalidMetadata	bool	false
ArrayEncryption	bool	false
TypeScrambler	bool	false
NumericObfuscation	bool	false
AntiMemoryDump	bool	false
BranchConfusion	bool	false
CodeVirtualization	bool	false
EntryPointMover	bool	false
MethodInliner	bool	false
AntiHook	bool	false
AntiHttp	bool	false
DnSpyCrasher	bool	false

15. Appendix B - File map

Where each protection lives in the source tree (paths are relative to **MasonProtectorI**).

Component	Path
Engine entry	Core/Obfuscation.cs
Settings model	Core/Engine/ProtectionSettings.cs
Pre-analysis	Core/PreAnalysis.cs
Polymorphic engine	Core/PolyEngine.cs
Builder UI	Builder.cs / Builder.Designer.cs
Program entry	Program.cs
Encryption (group)	Core/Protections/Encryption/*.cs
StringEncryption	Core/Protections/Encryption/StringEncryption.cs
IntEncoding	Core/Protections/Encryption/IntEncoding.cs
ConstantsEncoding	Core/Protections/Encryption/ConstantsEncoding.cs
FieldEncryption	Core/Protections/Encryption/FieldEncryption.cs
ArrayEncryption	Core/Protections/Encryption/ArrayEncryption.cs
MethodBodyEncryption	Core/Protections/Encryption/MethodBodyEncryption.cs
CrossReferenceEncryption	Core/Protections/Encryption/CrossReferenceEncryption.cs
PolymorphicEncryption	Core/Protections/Encryption/PolymorphicEncryption.cs
RuntimeEncryption	Core/Protections/Encryption/RuntimeEncryption.cs
MutationEncoding	Core/Protections/Encryption/MutationEncoding.cs
StringComposition	Core/Protections/Encryption/StringComposition.cs
DelegateEncryption	Core/Protections/Encryption/DelegateEncryption.cs
BodyVault / runtime / cloner	Core/Protections/Encryption/BodyVault.cs, BodyVaultRuntime.cs, TypeCloner.cs
Obfuscation (group)	Core/Protections/Obfuscation/*.cs
VM (V1 + V2 unified)	Core/Protections/Obfuscation/VMObfuscation.cs
ControlFlow / CFF2	Core/Protections/Obfuscation/ControlFlow.cs, ControlFlowFlattening2.cs
Opaque predicates	Core/Protections/Obfuscation/OpaquePredicate.cs
Proxy / ref / call hiding	Core/Protections/Obfuscation/ProxyCalls.cs, ReferenceProxy.cs, CallHiding.cs
Calli / Local2Field / Stack	Core/Protections/Obfuscation/CalliConversion.cs, Local2Field.cs, StackUnderflow.cs
Scattering / inliner	Core/Protections/Obfuscation/MethodScattering.cs, MethodInliner.cs
Numeric / branch / virt.	Core/Protections/Obfuscation/NumericObfuscation.cs, BranchConfusion.cs, CodeVirtualization.cs
Anti-* (group)	Core/Protections/Anti/*.cs
Debug + Scatter	Core/Protections/Anti/AntiDebug.cs, ScatterAntiDebug.cs
Tamper / Dump / MemDump	Core/Protections/Anti/AntiTamper.cs, AntiDump.cs, AntiMemoryDump.cs
Hook / Http / VM	Core/Protections/Anti/AntiHook.cs, AntiHttp.cs, AntiVM.cs
De4dot / ILDasm	Core/Protections/Anti/AntiDe4dot.cs, AntiILDasm.cs
Stealth (group)	Core/Protections/Stealth/*.cs

Component	Path
Renamer	Core/Protections/Stealth/Renamer.cs
Hide / Junk / Watermark	Core/Protections/Stealth/HideMethods.cs, JunkCode.cs, Watermark.cs
Fake attributes	Core/Protections/Stealth/FakeAttributes.cs
TypeScrambler / Token / Meta	Core/Protections/Stealth/TypeScrambler.cs, TokenConfusion.cs, MetadataConfusion.cs, InvalidMetadata.cs
DnSpyCrasher / EntryMover	Core/Protections/Stealth/DnSpyCrasher.cs, EntryPointMover.cs
Tooling	_tools/strip_comments.py, _tools/merge_vm.py, _tools/build_book.py, _tools/clean_testapp.py
Test harness	TestApp/V2NewobjProbe.cs, V1V2Probe.cs, AllOnExceptAntiTest.cs, TestApp.cs

End of MasonBook.

EIYM / Mason Protector - (c) Mason - Build edition 2026.04.